

Causal Assessment of BatchEvict, KV-Transfer Latency, and TTFT

Engineering conclusion

Mooncake source establishes a causal contention mechanism between BatchEvict and an ordinary L3 cache read. Eviction acquires metadata-shard mutexes in exclusive mode. BatchGetReplicaList acquires the occupied shards in shared mode. When both operations overlap on the same shard, the foreground lookup waits for the exclusive interval to complete.

The SGLang-observed KV-transfer metric surrounds the Mooncake storage-read operation. Its duration includes metadata lookup, shard-lock wait, replica selection, RDMA/TCP payload movement, completion, and validation. An eviction lock wait can therefore increase the reported transfer duration even when the payload transport itself is unchanged. The same delay can propagate into TTFT when the cache read remains on the request's critical path.

The available production metrics use 30-second samples, while an eviction cycle may complete in approximately one second. This aggregation combines many requests and multiple eviction phases in one observation. It does not resolve which request intersected an exclusive shard interval. The current telemetry therefore supports the source-derived causal mechanism but does not identify a production causal magnitude for KV-transfer latency or TTFT.

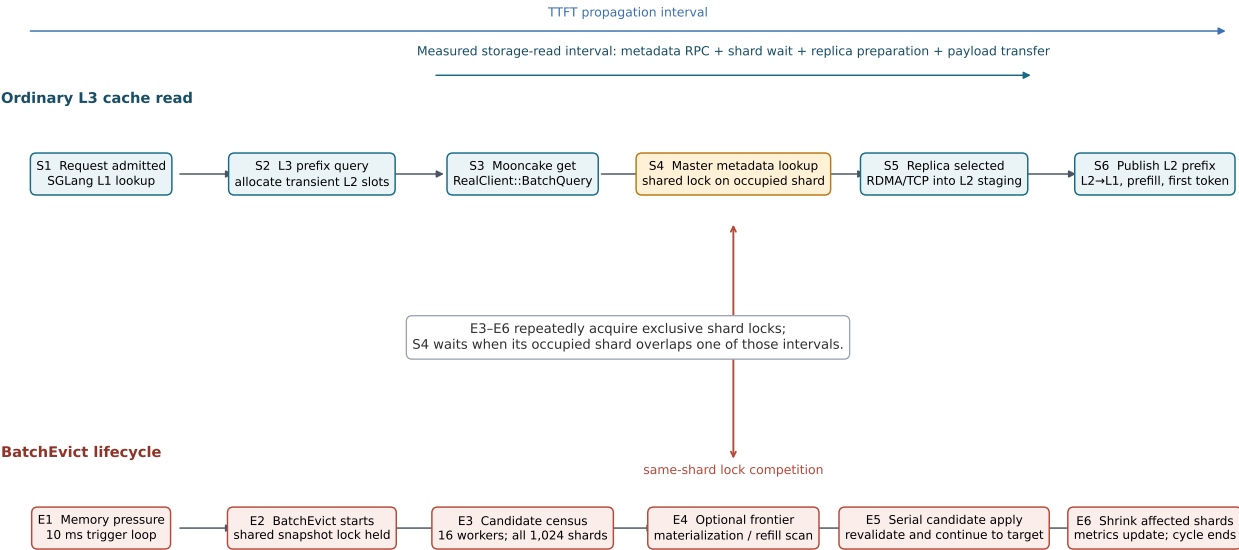
1. Deployment cache hierarchy

Mooncake is the main persistent cache for this deployment. SGLang supplies host-memory destination addresses for cache reads. These pages provide operation-scoped staging and publication storage:

```
Mooncake L3 MEMORY replica
-> RDMA/TCP into transient SGLang Host pages
-> publish the completed Host prefix
-> copy Host pages into L1 GPU pages
-> prefill and first token
```

Persistent cache responsibility begins at L3. Host memory provides the destination boundary required by the current Mooncake client interface. The adapter behavior was verified against the deployment guide's storage-read path and cache-hierarchy description.

SGLang-Mooncake cache-read path and BatchEvict contention boundary



Snapshot lock: shared for the full cycle and compatible with the read path. Shard lock: exclusive only while each shard or candidate is processed; up to 16 distinct shards during census.

2.1 Ordinary L3 cache-read lifetime

Stage	Execution	Latency contribution
S1	SGLang admits the request and performs its local L1 match.	TTFT begins upstream of storage access.
S2	SGLang queries the remaining L3 prefix and prepares Host destination slots.	Query, queueing, allocation, and prefetch policy influence request progress.
S3	batch_get_v1 supplies destination addresses; RealClient enters batch_get_into_multi_buffers and calls BatchQuery.	The Mooncake operation timer starts before metadata lookup.
S4	BatchGetReplicaList groups keys by shard, takes the snapshot lock in shared mode, and takes each occupied shard in shared mode.	Same-shard exclusive activity contributes shared-lock wait here.
S5	The client selects a replica and BatchGet transfers bytes through local copy, TCP, or RDMA.	Payload size, placement, transport queueing, and completion waiting contribute here.
S6	SGLang publishes the completed Host prefix, restores L1 pages, schedules prefill, and emits the first token.	Storage delay propagates into TTFT when the L3 read remains on the critical path.

`RealClient::batch_get_into_multi_buffers` starts `execute_timed_operation` before the internal call. The internal call performs `BatchQuery` before `BatchGet`. The outer interval therefore includes the metadata RPC and shard wait as well as payload transfer. See `real_client.cpp` and `core/architecture.md`.

2.2 BatchEvict lifetime

Stage	BatchEvict operation	Read-path interaction
E1	EvictionThreadFunc samples memory usage every 10 ms and reacts to the high watermark or allocation-pressure flag.	The background cycle starts independently of a foreground read.
E2	BatchEvict acquires snapshot_mutex_ in shared mode for the full cycle.	The read path also uses shared mode at this boundary.
E3	At most 16 census workers partition all 1,024 metadata shards; each worker visits its shards sequentially.	MetadataShardAccessorRW takes each visited shard exclusively.
E4	Selective ratios can trigger frontier materialization, recovery refill, or a second scan.	These scans revisit shards under exclusive access.
E5	Candidate application performs fresh lookup and revalidation under an exclusive shard lock and continues toward the target.	Revalidation creates further collision intervals.
E6	Sparse maps for affected shards are shrunk under exclusive access before the cycle completes.	The final exclusive intervals occur here.

The census covers 1,024 shards through at most 16 workers. Each worker holds one shard at a time, so the mechanism is shard-selective rather than a simultaneous exclusive lock over all shards. Candidate application later reacquires individual shard locks. Source locations include `master_service.h`, `BatchGetReplicaList`, `EvictionThreadFunc`, `BatchEvict`, and the detailed eviction architecture.

3. Causal path

For one successful L3 read:

```
Lstorage = Ladapter + Lmetadata-RPC + Lshard-wait
          + Ldescriptor + Lpayload-transfer + Lcompletion

TTFT = Ladmission-and-scheduling + Lstorage + LHost-to-L1 + Lprefill
```

The causal path is:

```
memory and allocation pressure
-> BatchEvict cycle
-> exclusive lock on shard s
-> overlapping BatchGetReplicaList lookup for shard s waits
-> SGLang-observed storage-read interval increases
-> TTFT increases when the storage read is on the critical path
```

3. Causal path

Factor	Causal role	Required observation
Eviction active interval	Defines treatment timing.	Cycle and phase start/end timestamps.
Shard intersection	Defines whether a read receives the lock-wait treatment.	Eviction shard and lookup shard identifiers.
Exclusive hold duration	Bounds direct metadata delay.	Per-shard exclusive acquisition, wait, and hold time.
Foreground shared-lock wait	Measures the direct mediator.	BatchGetReplicaList shared-lock wait per shard and request.
Payload transfer	Provides a parallel component of the composite metric.	Separate RDMA/TCP issue, queue, transfer, and completion timers.
Request critical path	Determines propagation into TTFT.	Request-level storage interval, scheduling phases, prefill, and first-token time.
Request pressure	Causes eviction and independently affects latency.	Arrival rate, concurrency, batch size, prompt tokens, hit length, and payload bytes.

4. Production observations and causal resolution

The causal treatment operates on an approximately one-second process and on individual shard-lock intervals within that process. A 30-second sample has a sampling-to-treatment ratio near 30:1. It aggregates the following populations:

- requests completed before the eviction cycle;
- requests overlapping the cycle on other shards;
- requests overlapping the same shard during an exclusive interval;
- requests completed after the cycle;
- latency changes produced concurrently by request pressure and transport.

The inferred event marker is a net key and memory contraction observed at the end of one dashboard bucket. It can occur after the lock-active census or candidate-application phase. This creates treatment-time uncertainty on the same order as the entire metric bucket.

Request pressure also acts as a common cause:

```
request pressure -> memory pressure -> BatchEvict  
request pressure -> scheduling and transport queueing -> latency
```

Consequently, correlation between the 30-second event marker and a latency percentile can combine causal lock wait, temporal dilution, shard exposure, queueing, and workload composition. A small coefficient can coexist with a real per-request lock wait, while a positive coefficient can also arise from the shared pressure driver. The aggregate series therefore provides a production-regime map. Causal-effect estimation uses the request- and shard-aligned design in Section 5.

4.1 Whole-range production status

The following figure covers the complete 12:00–06:00 observation range. Faint lines show each master's full export, while emphasized segments identify the master serving the client in the supplied attachment phase. Downward markers denote 30-second key-and-memory contraction signatures on the active master. The vertical boundary and intervening metric gap locate the transition from the shared master to the independent master.

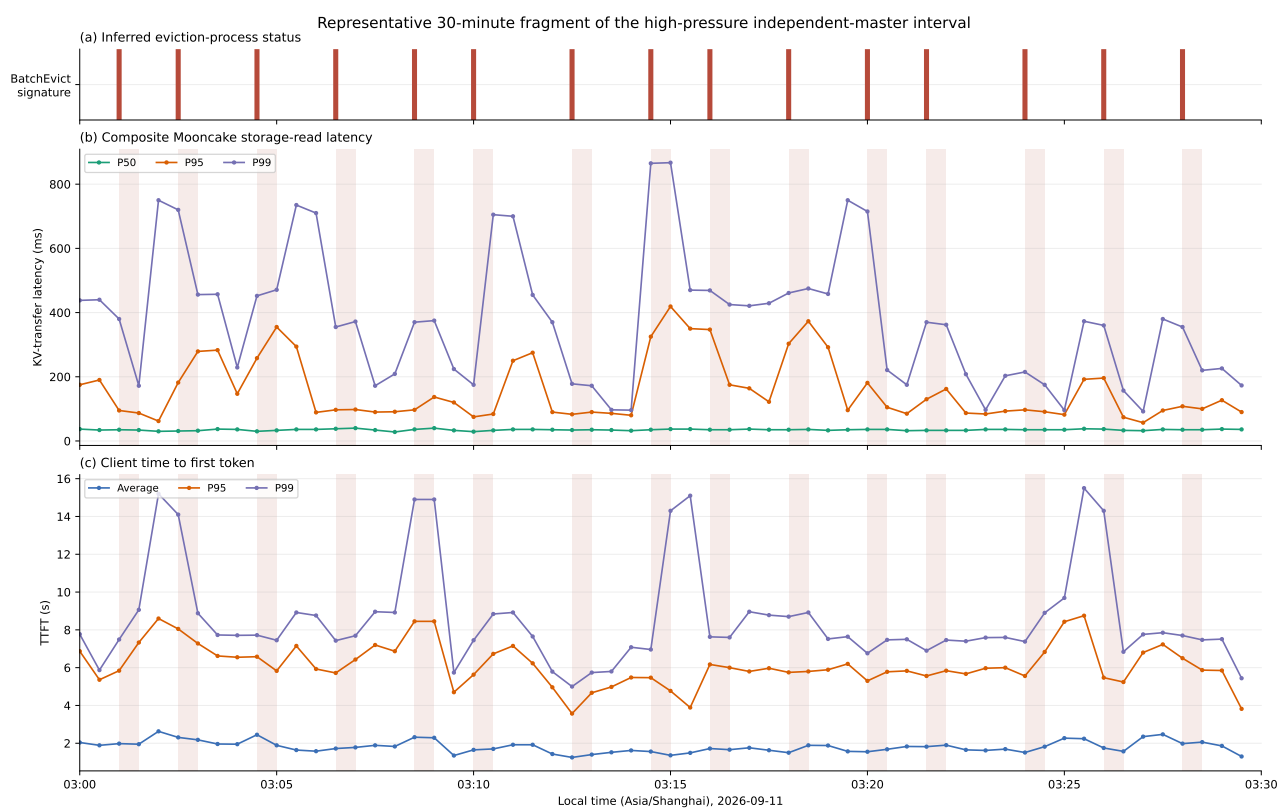
Mooncake client transition and production latency, 30-second observations



The phase summary records 13.0 inferred signatures per hour while the client uses the shared master and 22.3 per hour while it uses the independent master. Median KV-transfer P95 is 466 ms and 91 ms, respectively; median TTFT P99 is 5.80 s and 7.43 s. These values characterize two distinct production regimes that differ in master sharing, cache growth, request mix, and pressure. The whole-range view supplies deployment context, while the high-pressure fragment and within-regime association below provide the event-focused observations.

4.2 Descriptive high-pressure fragment

The integrated view below retains the 03:00-03:30 fragment from the sustained high-pressure interval. Red bands mark 30-second samples containing the inferred key-and-memory contraction signature. The latency panels show the raw composite Mooncake storage-read metric and client TTFT on the same time axis.

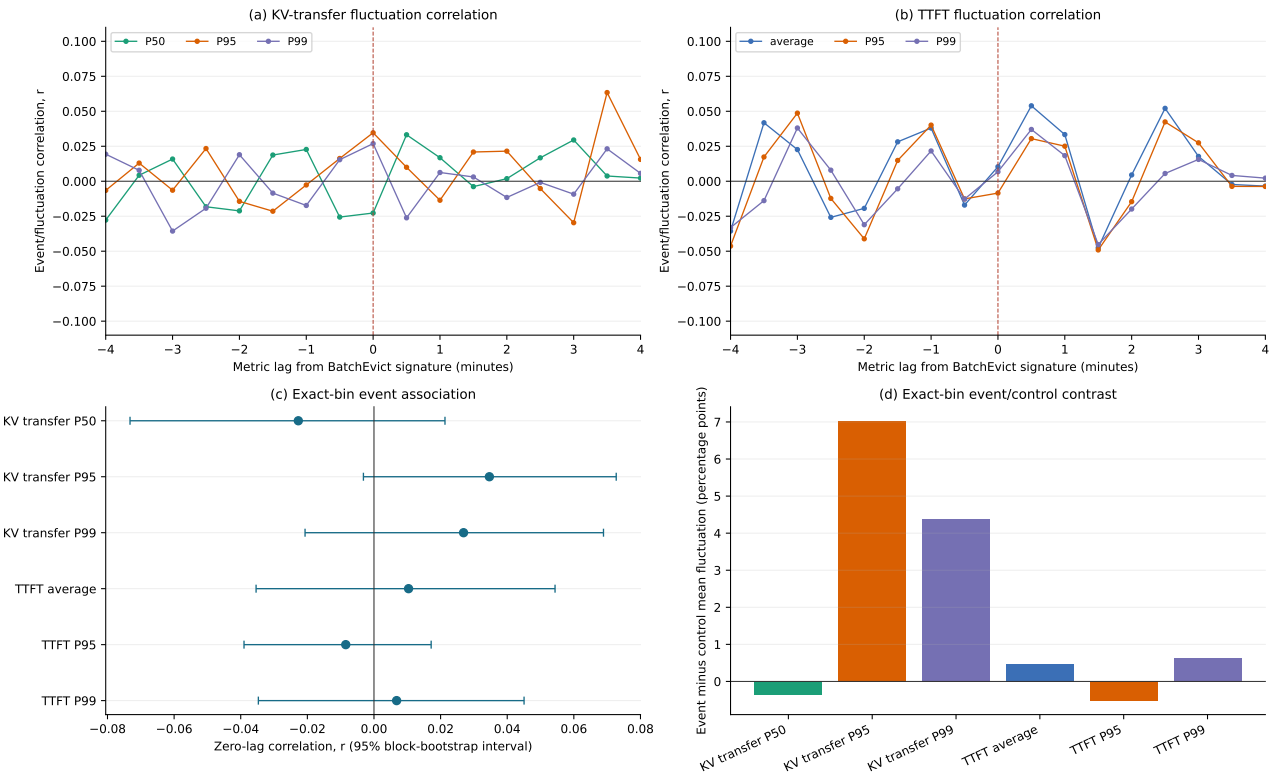


This view documents temporal co-occurrence and the scale of production fluctuation. Each red band is an aggregate event marker rather than the exact approximately one-second lock-active interval, so causal inference continues to use the request- and shard-aligned design below.

4.3 Resolution-limited event association

The event-association diagram uses all 1,201 samples from the 20:00–06:00 high-pressure interval. Each latency metric is expressed as deviation from a centered 30-minute local median, and the inferred BatchEvict marker is shifted from -4 through $+4$ minutes. Thirty-minute block resampling supplies the displayed uncertainty intervals.

High-pressure interval: BatchEvcit correlation with local latency fluctuations



At the inferred event bucket, KV-transfer P95 has correlation $+0.035$ with a 95% block-bootstrap interval of -0.003 to $+0.073$. KV-transfer P99 has correlation $+0.027$ with an interval of -0.021 to $+0.069$. TTFT correlations remain near zero: $+0.010$ for average, -0.008 for P95, and $+0.007$ for P99.

These values describe small aggregate co-variation at 30-second resolution. The intervals span zero, the lead/lag curves change sign, and the inferred event bucket can follow the approximately one-second lock-active phase. This pattern is consistent with a low-rate or shard-local latency effect diluted by bucket aggregation. The diagram serves as production association evidence; causal-effect estimation is reserved for request- and shard-aligned measurements.

5. Production causal validation

The identifying comparison is between foreground reads that overlap an exclusive eviction lock on their own shard and comparable reads that do not. The unit of analysis is a request-shard lookup rather than a 30-second bucket.

5. Production causal validation

Instrument	Dimensions	Causal estimate enabled
BatchEvict cycle and phase timers	master, cycle, census/frontier/apply/shrink	Exact treatment interval and phase.
Exclusive shard-lock timer	master, shard, cycle, phase	Eviction-side wait and hold duration.
Shared lookup-lock timer	master, shard, request, batch	Direct foreground contention delay.
Metadata RPC timer	client, request, batch	End-to-end master component.
Payload timer	client, request, bytes, transport, source	RDMA/TCP component separated from master wait.
SGLang request timeline	request, hit pages, prompt tokens, concurrency	Propagation from storage delay into TTFT.

The analysis population is divided into:

- treated lookups: the request shard intersects an exclusive eviction interval;
- concurrent controls: the request overlaps the same cycle on another shard;
- temporal controls: matched requests immediately before or after the cycle.

Matching or regression adjustment uses request bytes, hit length, prompt tokens, transport, source node, concurrency, and queue depth. Concurrent other-shard controls hold master load and cycle timing approximately constant, while the treated comparison isolates shard intersection.

The direct storage effect is:

$$\Delta L_{\text{storage}} = E[L_{\text{storage}} \mid \text{same-shard exclusive overlap}, X] \\ - E[L_{\text{storage}} \mid \text{concurrent other-shard lookup}, X]$$

The mediator check is:

$$\Delta L_{\text{storage}} \approx \Delta L_{\text{shared-lock-wait}}$$

The TTFT propagation effect is evaluated on the same request:

$$\Delta \text{TTFT} = E[\text{TTFT} \mid \text{measured shard-lock wait}, X] \\ - E[\text{TTFT} \mid \text{matched request without shard-lock wait}, X]$$

Here, X contains the matched workload, transport, and scheduling variables. Causal confirmation requires three aligned observations: exclusive eviction occupancy, foreground shared-lock wait on the same shard, and an increase in the enclosing storage-read interval. Request-level TTFT then measures the downstream propagation coefficient.

6. Production claim boundary

The source supports the following mechanism claim:

A BatchEvict cycle can increase the SGLang-observed Mooncake storage-read duration when an ordinary metadata lookup overlaps an exclusive eviction interval on the same shard.

The present 30-second telemetry supports deployment context and metric-range description. Its resolution does not support a mutex-specific causal magnitude or a causal TTFT estimate. The request- and shard-aligned design in Section 5 provides the required production confirmation.

Reproducibility

The source-file map is stored in data/README.md. The commit-ready PDF is generated from this Markdown source by render_analysis_pdf.py.